DECORATORS: THE BASICS

A **decorator** is a function that wraps another function and adds behaviour to it.

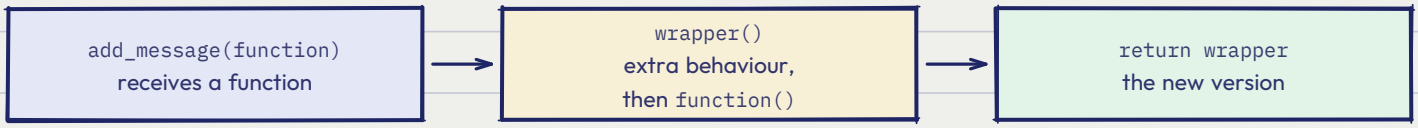
1) Wrap a function by hand

Example :

```
def add_message(function):
    def wrapper():
        print("Function is starting...")
        function()
    return wrapper
def greet():
    print("Hello, students!")
greet_with_message = add_message(greet)
greet_with_message()
```

Output :

```
Function is starting...
Hello, students!
```



greet itself is not rewritten, the decorator adds something around it

2) The @ shortcut

Example :

```
def add_message(function):
    def wrapper():
        print("Function is starting...")
        function()
    return wrapper
@add_message
def greet():
    print("Hello, students!")
greet()
```

Output :

```
Function is starting...
Hello, students!
```

@add_message above the function is the same as writing greet = add_message(greet)

Reusing & Extending Decorators

One decorator can wrap many functions, pass along arguments, and even remember state.

1) One decorator, two functions

Example :

```
def announce(function):
    def wrapper():
        print("--- Starting task ---")
        function()
    return wrapper
@announce
def make_tea():
    print("Making tea.")
@announce
def study_python():
    print("Studying Python.")
make_tea()
study_python()
```

Output :

```
--- Starting task ---
Making tea.
--- Starting task ---
Studying Python.
```

2) Arguments and a return value

Example :

```
def announce(function):
    def wrapper(*args, **kwargs):
        print("Running function...")
        return function(*args, **kwargs)
    return wrapper
@announce
def add_numbers(a, b):
    return a + b
print("Answer:", add_numbers(10, 20))
```

Output :

```
Running function...
Answer: 30
```

3) Count the calls

Example :

```
def count_calls(function):
    calls = {"count": 0}
    def wrapper(*args, **kwargs):
        calls["count"] += 1
        print("Call number:", calls["count"])
        return function(*args, **kwargs)
    return wrapper
@count_calls
def say_hi(name):
    print("Hi,", name)
say_hi("Rahul")
say_hi("Priya")
```

Output :

```
Call number: 1
Hi, Rahul
Call number: 2
Hi, Priya
```

Mistakes & Constructors

Avoid the common decorator slips, then revise `__init__()` with `self`.

1) Common decorator mistakes

Mistake	Fix
forgetting return wrapper	the decorator must return the wrapper
writing function instead of function()	add the brackets to call it
@decorator below the function	place it directly above the def
not returning the result	return function(...) inside the wrapper

2) Constructors

Constructor

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age
student1 = Student("Rahul", 20)
student2 = Student("Priya", 21)
print(student1.name, student1.age)
print(student2.name, student2.age)
```

Output :

```
Rahul 20
Priya 21
```

Default argument

```
class Employee:
    def __init__(self, name, department="Not assigned"):
        self.name = name
        self.department = department
employee1 = Employee("Amit")
employee2 = Employee("Neha", "Data Science")
print(employee1.name, "-", employee1.department)
print(employee2.name, "-", employee2.department)
```

Output :

```
Amit - Not assigned
Neha - Data Science
```

Constructor plus a method

```
class Rectangle:
    def __init__(self, length, width):
        self.length = length
        self.width = width
    def area(self):
        return self.length * self.width
rectangle = Rectangle(5, 3)
print("Area:", rectangle.area())
```

Output :

```
Area: 15
```

the constructor sets up the data; a method uses that data to do a task

Practice & Revision

Write each one yourself first, then compare. Each solution is run for real.

Task 1 add_welcome decorator.

Solution :

```
def add_welcome(function):
    def wrapper():
        print("Welcome to Python!")
        function()
    return wrapper
@add_welcome
def start_class():
    print("Class has started.")
start_class()
```

```
Welcome to Python!
Class has started.
```

Task 2 Decorator with return value.

Solution :

```
def announce(function):
    def wrapper(*args, **kwargs):
        print("Calculating...")
        return function(*args, **kwargs)
    return wrapper
@announce
def multiply(a, b):
    return a * b
print("Answer:", multiply(4, 5))
```

```
Calculating...
Answer: 20
```

Task 3 Decorator and marks.

Solution :

```
def report_message(function):
    def wrapper(*args, **kwargs):
        print("Generating marks report...")
        return function(*args, **kwargs)
    return wrapper
@report_message
def average_marks(marks):
    return sum(marks) / len(marks)
print("Average:", average_marks([80, 90, 70]))
```

```
Generating marks report...
Average: 80.0
```

Task 4 Car constructor.

Solution :

```
class Car:
    def __init__(self, brand, color):
        self.brand = brand
        self.color = color
car1 = Car("Toyota", "White")
car2 = Car("Honda", "Black")
print(car1.brand, car1.color)
print(car2.brand, car2.color)
```

```
Toyota White
Honda Black
```

Task 5 Decorator plus constructor.

Solution :

```
def announce(function):
    def wrapper(*args, **kwargs):
        print("Creating student record...")
        return function(*args, **kwargs)
    return wrapper
class Student:
    def __init__(self, name, course):
        self.name = name
        self.course = course
@announce
def show_student(name, course):
    student = Student(name, course)
    print("Name:", student.name)
    print("Course:", student.course)
show_student("Rahul", "Python Programming")
```

```
Creating student record...
Name: Rahul
Course: Python Programming
```